Homework for Class 1: Python Fundamentals

1. Variables, Data Types & Dynamic Typing

Problem 1.1: Type Casting & Truthiness Evaluation

Create a function or script that receives three raw inputs:

- `raw_price`: string (e.g., `"199.99"`)
- `raw_quantity`: string (e.g., `"5"`)
- `raw_discount`: string (e.g., `"0"`)

Requirements:

1. Cast `raw_price` to a `float` and `raw_quantity` to an `int`.
2. Compute `total_cost = price * quantity`.
3. Evaluate the truthiness of `raw_discount`. If `raw_discount` is non-zero, calculate a discounted price; otherwise, keep `total_cost` as is.
4. Verify whether `total_cost is None` vs `not total_cost`. Write comments explaining the difference between these two checks.

---

2. String Formatting & Manipulation

Problem 2.1: Dynamic Prompt Generator

Build a string-processing function `generate_llm_prompt()` that formats incoming system instructions and dynamic user inputs for an AI API.

Requirements:

1. Accept three parameters: `system_role` (str), `user_query` (str), and `temperature` (float).
2. Clean `user_query` by removing leading/trailing whitespaces.

3. Use an **f-string** with specific formatting specs:

- Format `temperature` to exactly 2 decimal places (e.g., `0.70`).
- Convert `system_role` to uppercase.

1. Capitalize the first letter of `user_query`.

Expected Input:

```
generate_llm_prompt("  system assistant  ", "   what is rag?  ", 0.7)
```

Expected Output:

```
[SYSTEM: ASSISTANT | TEMP: 0.70] What is rag?
```

---

3. Control Flow: If-Else & Loops

Problem 3.1: API Retry Counter with Exponential Backoff

Simulate an API request retry handler using a loop.

Requirements:

1. Given a list of simulated server responses: `responses = [500, 502, 503, 200, 404]`.
2. Iterate through `responses` using a `while` loop or `for` loop:

- If the code is `500`, `502`, or `503`, print a warning message indicating a retry, increment the attempt counter, and double the wait time (start backoff at 1 second).
- If the code is `200`, print success and `break` out of the loop immediately.
- For any other error (e.g., `404`), print a fatal error and `break`.

1. Skip `None` or invalid values in the list using `continue`.

---

4. Core Collections

Problem 4.1: Message Deduplication & Inverted Index

Write a script that processes a list of raw chat messages to deduplicate tags and index messages by user.

Input Data:

```
chat_logs = [
    {"user": "Alice", "tags": ["python", "ai"], "msg": "Hello world"},
    {"user": "Bob", "tags": ["rag", "ai"], "msg": "Building a RAG pipeline"},
    {"user": "Alice", "tags": ["python"], "msg": "Learning collections"},
]
```

Requirements:

1. Create a `set` containing all **unique tags** across all chat messages.
2. Build a `dict` mapping each `user` to a `list` of their messages.
3. Access missing keys safely using `.get()` without raising a `KeyError`.

Expected Outputs:

```
# Unique Tags Set:
{"python", "ai", "rag"}

# User Index Dict:
{
    "Alice": ["Hello world", "Learning collections"],
    "Bob": ["Building a RAG pipeline"]
}
```

---

5. Functions, Type Hints & Mutable Defaults

Problem 5.1: The Safe System Log Collector

Implement a function that collects log records for an AI service.

Requirements:

1. Define a function `add_log_entry()` with type hints for parameters and return values:

- `event`: string
- `level`: string (default: `"INFO"`)
- `logs`: optional list of strings (default: `None`)

1. Avoid the **mutable default argument trap** (do NOT use `logs=[]` as the default argument).
2. If `logs` is `None`, instantiate a new list inside the function.
3. Append a timestamped/formatted log string to `logs` and return the list.

Code Skeleton:

```
def add_log_entry(event: str, level: str = "INFO", logs: list[str] = None) -> list[str]:
    """
    Safely adds a log entry to the log list without mutable default bugs.
    """
    # TODO: Implement safe default initialization and append logic
    pass
```